Multi-tiered filesystem policies

Kathir Meyyappan

Johnston Liu

University of Chicago

CMSC 33100

Motivation

- Storage gets more expensive as it gets faster, so we have less fast storage and more slow storage in most systems.
  - Large storage systems leverage indirection to manage data in a seamless manner.
  - Policies to shuffle around this data are extremely important depending on the workload, i.e. high-write object store systems.
- Cloud storage services like EFS use distributed storage
  - Shard files, maintain symlinks, replicate data, etc.
- But what if we have single machine constraints (e.g. security-concerned use-cases)?
  - Unrealistic to maintain 1 large drive, even if a local workload calls for this.
  - Solutions like ZFS tiering exist, but those are not focused on file location, but rather block-level movement, which has much more OS overhead.

Realistically…

- This isn't much of a concern in day-to-day systems, and there are are always secure cloud-native workarounds that may cost more money.
  - But still… **cheap = good.**
- And we still might care for field-deployed systems, air-gapped research workstations, etc.
- How efficient can we get on one machine with purely filesystem level symlinking?
  - And do policies are used for other cases (page replacement, thread scheduling, cache, etc.) transfer well to file system hierarchy management?

The goal

- Have a user that works in some file space `./hot/` that has **limited size**.
  - This means that entire working fileset can't fit directly in directory.
  - Symlinks are mandatory if we want to abstract this lack of space away from user.
- Policy engine (daemon thread) dynamically defers file contents to other directories (e.g. `./cold/`) based on access patterns so that:
  - We incur as little data movement (i.e. cost and latency) as possible.
  - The hit rate in our warmer directories is as high as possible (for low latency).

Program specifications

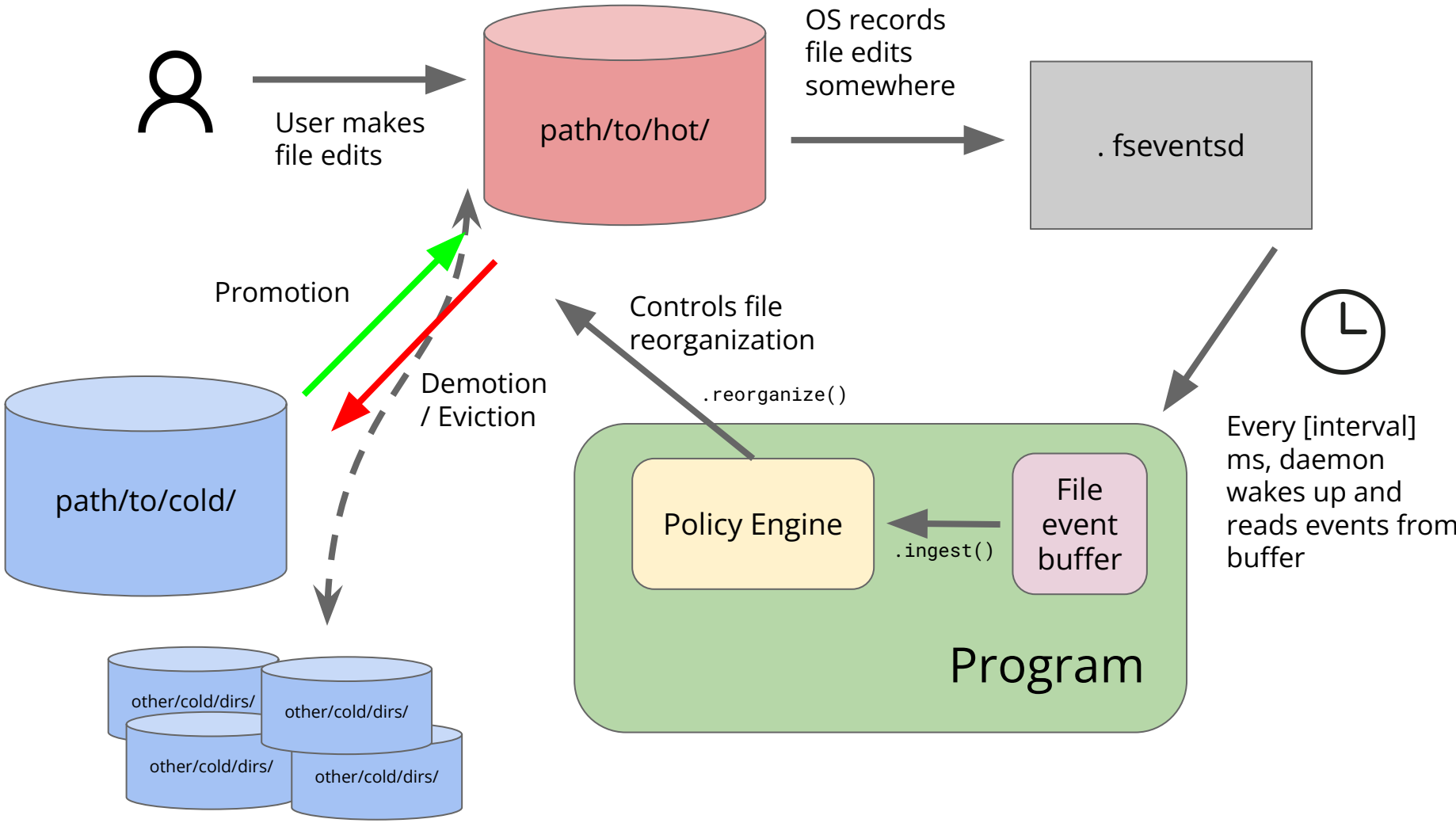
- Frequently access files stored in “hot” drive
- Stale or infrequent access files are stored in “cold” drive (in a generalized model, we have many levels of cold drives)
- Policy engine will determine how files move between these layers.
  - Will be based on some facets of user interaction with filesystem, in any case.
  - Engine will ideally make it so that file movement is low, hit rate in warmer directories is higher, etc.

A few implementation details

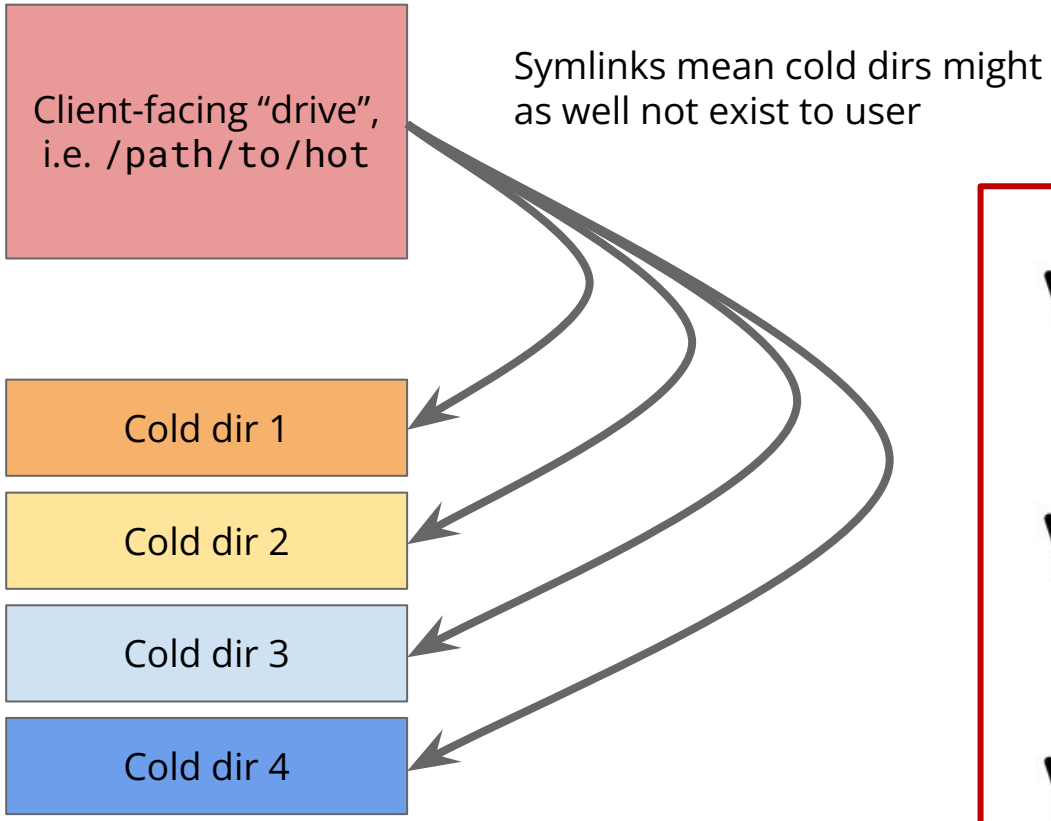
https://github.com/kathirmeyyappan/filestore-tiering

- **PolicyEngine**
  - Modular policies with internal data structures to make reorganization decisions
  - Uses TierState interface to move files and record stats
  - Program is set up so that new policies are very easily added - just inherit from PolicyEngine class, implement methods, and drop it in.
- **Watcher class**
  - main() runs a daemon thread that wakes up every once in a while and drains a buffer that has been pulling from the file edits pool (FSEvents for MacOS).
- Bunch of other stuff (benching engine, non-intrusive logging modules, etc.)

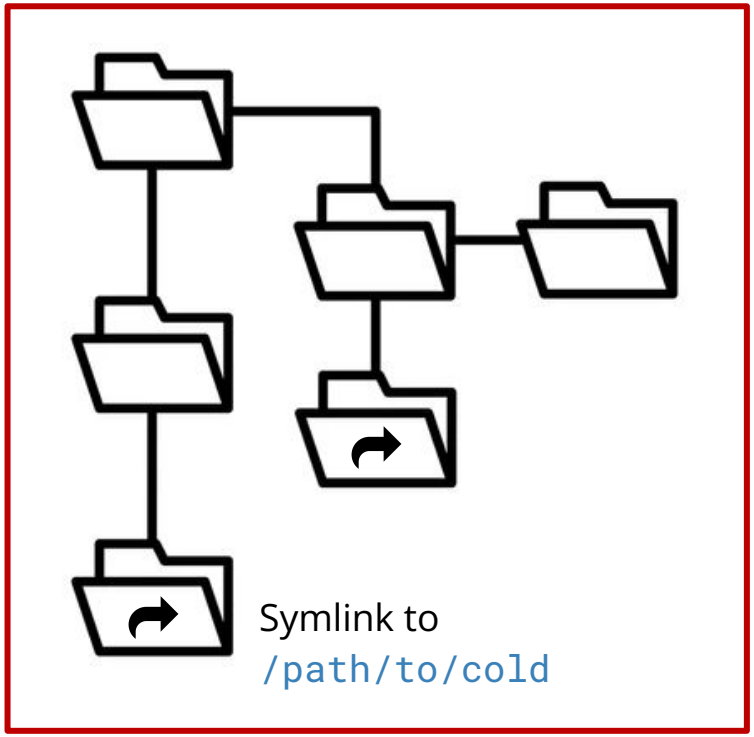
High-level design



Storage Hierarchy



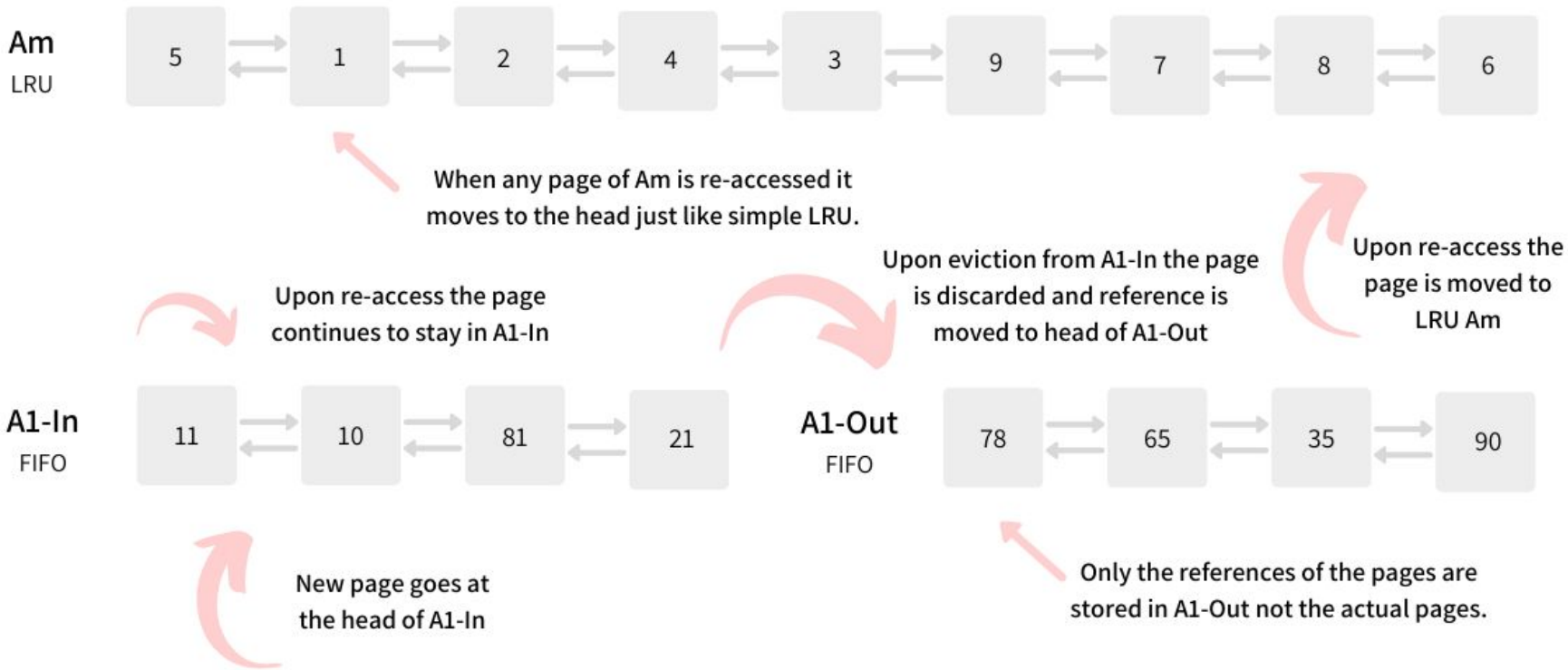
path/to/hot/



Policies

- LRU
- LFU
- LRU_2Q
- ARC
- CACHEUS
- LeCaR

LRU_2Q



2Q - Full Version

ARC

- ARC (Adaptive Replacement Cache) balances recency and frequency using four lists and one adaptive parameter p (Megiddo & Modha, FAST '03)

- Maintain two lists in the hot tier: T1 for files seen once (recency) and T2 for files seen more than once (frequency), with ghost lists B1/B2 tracking recently evicted files from each

- When a ghost-listed file is re-accessed, ARC shifts the T1/T2 balance toward whichever list lost that file and automatically adapting to favor recency or frequency based on the workload

LeCaR

- LeCaR (Vietri et al., HotStorage '18) maintains two eviction experts (LRU and LFU) and picks between their victims via a **weighted coin flip**, learning which expert to trust through online regret minimization

- Evicted files go to ghost lists tagged with which expert chose them; re-access penalizes that expert and shifts weight toward the other, with recent mistakes penalized more heavily

CACHEUS

- CACHEUS extends LeCaR with three improvements (Gil et al., FAST '21)
- Replaces LRU with SR-LRU (Scan-Resistant LRU): new files enter a probationary FIFO; only on second access are they promoted to a protected LRU queue. One-hit scan files get evicted first
- Replaces LFU with CR-LFU (Cache-Resident LFU): frequency counts decay exponentially `freq = freq * decay_factor + 1` preventing stale popular files from permanently occupying the cache
- Uses an adaptive learning rate: `lr = max(0.001, lr_init * |2w - 1|)`. Learns slowly when uncertain (weight near 0.5), quickly when committed
- Same ghost list and weighted coin-flip eviction framework as LeCaR

Decision Tree

- Inspired by LRB (Song et al., NSDI '20) (their application is CDN caching), uses a decision tree to learn which files to evict, adapted for lightweight daemon application
- Combines four complementary features (recency, frequency, size, inter-access time) into a single learned scoring function
- Periodic retraining allows the model to adapt to workload shifts over time
- Ghost list tracks evicted files. Re-access means the eviction was wrong (label 1.0). No re-access means it was right (label 0.0).

Benchmarking

- Recorded things like:
  - Cache Hit Rate (%)
  - Bytes Promoted
  - Bytes Demoted
  - `.ingest()` latency
  - `.reorganize()` latency
- Seeded (creates / edits / deletes) operations and synchronously ran policies on file modification workloads.

Presets

- Workloads take in various parameters such as skew, hot_capacity, min_file_size, measure_ops, etc.
- Presets with set parameters were created, and we generated a prior for which policies might succeed.
  - **steady_state**: uniform random access, no pattern to exploit. Baseline where all policies should perform similarly
  - **frequency_favored:** edits concentrated on a small set of old files Rewards frequency-aware policies
  - **recency_favored**: high file creation rate, edits land on newest files. Rewards recency-aware policies
  - **high_churn**: heavy create+delete traffic constantly shifts which files exist. Tests adaptation to a moving working set

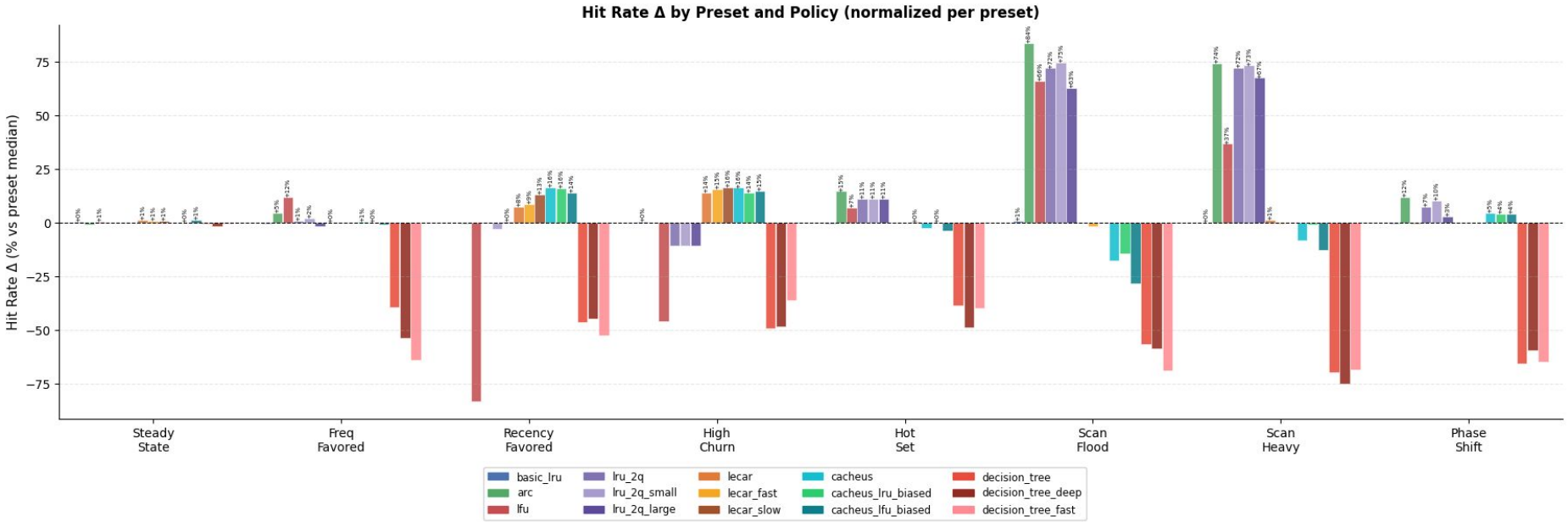
Presets (continued)

- **hot_set**: moderate frequency skew (skew=3.0). A milder version of frequency_favored
- **scan_flood**: many one-time file creates mixed with a stable frequently-accessed core. Tests scan resistance
- **scan_heavy**: extreme version of scan_flood with more creates and stronger skew. LRU-2Q's FIFO probation queue excels here
- **phase_shift**: alternates between frequency-dominated and recency-dominated access patterns. Tests how quickly policies adapt to workload changes

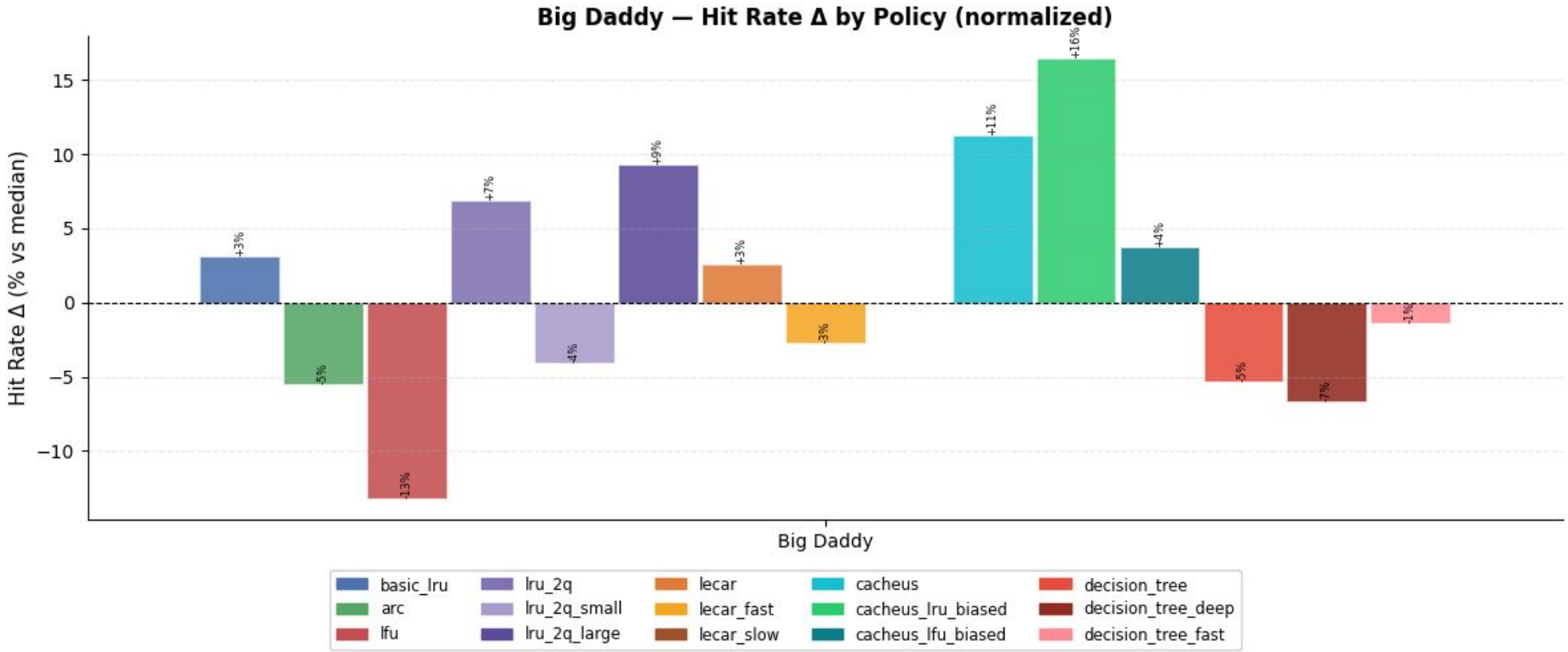
Throughput as a benchmark

- Throughput is incompatible with our policy evaluation.
  - Policies that move files look slower because reorganization runs synchronously and adds wall-clock overhead.
  - E.g. a “do nothing” policy maximizes throughput but achieves a 0% hit rate (optimizes the wrong objective).
- In production, reorganization cost is invisible — the daemon runs asynchronously and doesn’t affect app I/O paths.
  - What matters is placement quality (keeping the right files hot), not the daemon’s raw throughput.

Hit rate on all policies, various workloads

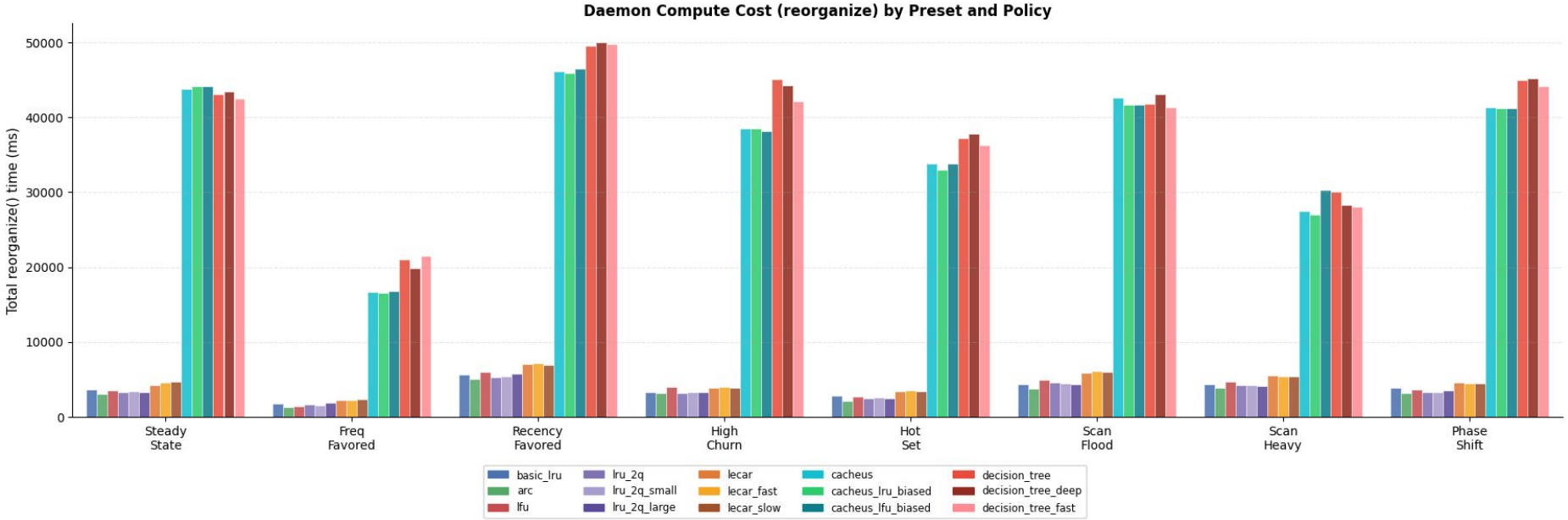


Mixed workload comparison

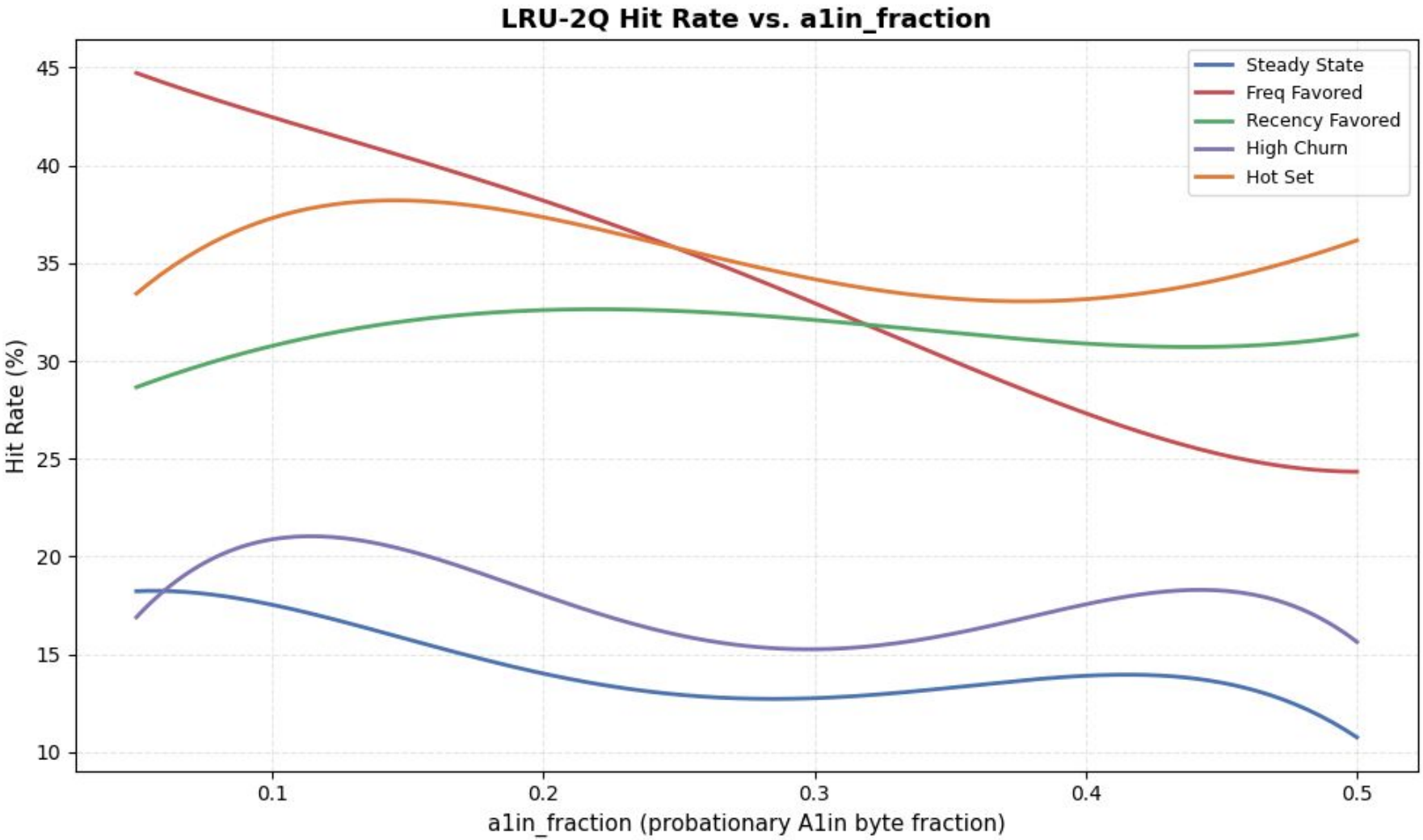


Byte write rates

Daemon latency



Tunable parameter progression



Conclusion

- Filesystem tiering is... hard.
  - There is no clear winner with simple policies and we seem to be largely dependent on the type of workload
- Program works though.
  - Will work out-of-the box when the only requirement is keeping all direct links in `./hot/` and there is a space limit there.
- Next step is multi-tiering, which is also hard.
  - But maybe we will see superior hit rates if we can think of a good metric for measuring it across varying-priority directories.